

005 gihyeon code review

다시 가져온 던파듀얼 탭 UI

배경

상황

캐릭터 선택 씬에는 속성 탭이 3개 있다.

- 큐브 선택 (30% / 50%)
- 캐릭터 컬러 선택
- 헬퍼 선택

탭이 서로를 직접 참조하며 전환을 처리하면 결합도가 높아지고, 탭 추가·유지보수가 어려워진다.

해결 방안

각 탭을 공통 인터페이스 `IAttributeTab` 으로 분리하고, `Select_AttributePanel` 이 현재 활성 탭과 전환 로직을 관리한다.

→ 탭 간 직접 의존 제거, 결합도 하락, 확장 가능한 구조

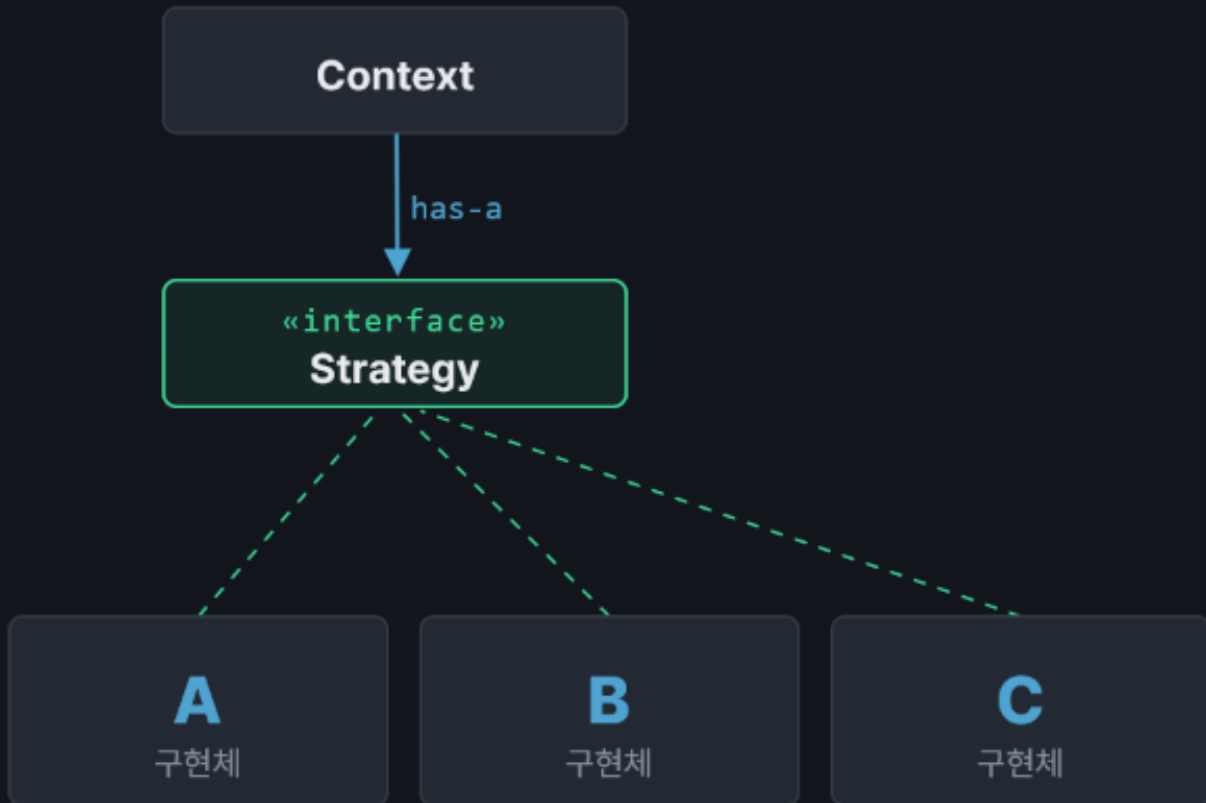


Strategy 패턴이란

알고리즘을 인터페이스로 캡슐화해
런타임에 교체 가능하게 만드는 패턴

- GoF(Design Patterns 저자 4인) 23개 패턴 중 행동(Behavioral) 패턴
- 구조: Context가 Strategy 인터페이스를 has-a로 보유, ConcreteStrategy들이 이를 구현
- if/else · switch 분기가 흩어지는 것을 막고 개방-폐쇄 원칙(OCP)을 지키기 위해 사용

패턴 구조 - ConcreteStrategy × 3





프로젝트 적용 — IAttributeTab

6개 가상 함수가 이 UI의
Strategy 인터페이스다

- IAttributeTab.h가 정의하는
Strategy 인터페이스
- Update, Late_Update, Render, Handle_Input, Set_Context, Initialize

```
IAttributeTab.h

class IAttributeTab abstract
{
    virtual HRESULT Initialize() = 0;
    virtual void Update(_float dt) = 0;
    virtual void Late_Update(_float) = 0;
    virtual HRESULT Render() = 0;
    virtual void Handle_Input() = 0;
    virtual void Set_Context(...) = 0;
};
```

모든 ConcreteStrategy가 이 6개 함수를 구현해야 한다.

프로젝트 적용 — ConcreteStrategy 3종

큐브 · 캐릭터 컬러 · 헬퍼, 세 탭이 각자 독립적으로 IAttributeTab을 구현한다.

CUBE



CAttributeTab_Cube

- 30 / 50 큐브 선택 처리
- 캐릭터별 3개 큐브 JSON 로드
- 반환값: CUBE_TYPE

W/S로 30 ↔ 50 큐브 전환

COLOR



CAttributeTab_CharColor

- 캐릭터 컬러 슬롯 처리
- 슬롯 4개, 스크롤바로 탐색
- 반환값: uint 컬러 인덱스

WASD로 포커스 프레임 이동

HELPER



CAttributeTab_Helper

- 헬퍼 슬롯 처리
- 2명 선택 (Left / Right)
- 반환값: SelectedHelperIcon 2개

Enter로 순차 2명 확정



프로젝트 적용 — Context

포인터 하나만 바꿔서 전
락을 전환한다

↻ CSelect_AttributePanel — 탭
3개를 미리 다 만들어 보관

🎵 Set_TabNum()이
포인터만 교체

탭 전환해도 내부 상태(선택 중이던 타입,
스크롤 위치)가 보존됨



Select_AttributePanel.cpp

```
void Set_TabNum(_uint iTabNum)
{
    if (iTabNum >= m_vecTabs.size())
        return;

    m_iCurTabIndex = iTabNum;
    m_pCurTab = m_vecTabs[iTabNum].get();
}
```

새 객체를 만들지 않으므로 각 탭이 갖고 있던 상태(선택 중이던 타입,
스크롤 위치 등)가 그대로 유지된다.

Level_Select에서 Tab을 생성하고 Panel에 전달하여 초기화

```
vector<unique_ptr<IAttributeTab>> tabs;

tabs.push_back(AttributeTab_Cube::Create());
tabs.push_back(AttributeTab_CharColor::Create());
tabs.push_back(AttributeTab_Helper::Create());

shared_ptr<UI> pPanel = m_pGameInstance->
    Find_UIGameObject(TEXT("CharAttributes_Base"), LEVEL_SELECT);

m_pAttributePanel = static_pointer_cast<Select_AttributePanel>(pPanel);
m_pAttributePanel->Set_AttributeTabs(std::move(tabs));
```

현재 활성 Tab에 따라 동작이 달라지며 Panel을 통해 입력과 상태 전환 처리

```
void Level_Select::Select_Cube_1P()
{
    if (m_pAttributePanel->Get_CurTab() !=
        m_pAttributePanel->Get_Tab(Select_AttributePanel::TAB_ORDER::CUBE))
        return;

    m_pAttributePanel->Handle_Tabs_Input();

    if (m_pGameInstance->Key_Down(DIK_RETURN))
    {
        auto* pCubeTab = static_cast<AttributeTab_Cube*>
            (m_pAttributePanel->Get_Tab(Select_AttributePanel::TAB_ORDER::CUBE));

        m_tBattle_Level_Desc.iLeftCharCube = pCubeTab->Get_SelectedCubeType();
        pCubeTab->Set_CubeSelected();
    }
}
```

Select_AttributePanel의 함수 일부

```
void Select_AttributePanel::Set_AttributeTabs(vector<unique_ptr<IAttributeTab>>&& vecTabs)
{
    m_vecTabs = std::move(vecTabs);

    if (!m_vecTabs.empty())
    {
        m_iCurTabIndex = 0;
        m_pCurTab = m_vecTabs[0].get();
    }
}

void Select_AttributePanel::Update(_float fTimeDelta)
{
    __super::Update(fTimeDelta);
    m_pCurTab->Update(fTimeDelta);
}
```

외부에서 만들어진 Tab을 받아와
Panel이 수명 관리 및 상호작용 관리

현재 활성 Tab만 Update해 해당 Tab의 로직 실행

Select_AttributePanel의 헤더 일부

```
class Select_AttributePanel final : public UI
{
public:
    enum TAB_ORDER { CUBE, CHARCOLOR, HELPER, TAB_END };

private:
    vector<unique_ptr<IAttributeTab>> m_vecTabs;
    IAttributeTab* m_pCurTab = nullptr;
    _uint m_iCurTabIndex = 0;

public:
    IAttributeTab* Get_Tab(TAB_ORDER order) const;
    IAttributeTab* Get_CurTab() const { return m_pCurTab; }

    void Set_TabNum(_uint iTabNum);
    void Set_AttributeTabs(vector<unique_ptr<IAttributeTab>>&& vecTabs);
};
```

Slide 3-3. Context는 구체 타입을 모른다

```
void CSelect_AttributePanel::Update(_float fTimeDelta)
{
    __super::Update(fTimeDelta);
    m_pCurTab->Update(fTimeDelta);    // 어느 탭인지 모른 채 위임
}
```



Select_AttributePanel.cpp 의 include는 IAttributeTab.h 하나뿐. 구체 탭 헤더를 한 번도 include 하지 않는다.

Slide 3-4. 인터페이스가 없었다면?

```
// — 인터페이스 없이 찼다면 Panel이 이렇게 됐을 것 —
void CSelect_AttributePanel::Update(_float fTimeDelta)
{
    if (m_iCurTabIndex == 0)    m_pCubeTab->Update(fTimeDelta);
    else if (m_iCurTabIndex == 1) m_pCharColorTab->Update(fTimeDelta);
    else if (m_iCurTabIndex == 2) m_pHelperTab->Update(fTimeDelta);
}
```



문제 3가지

1. 탭 추가 시 Update / Late_Update / Render / Handle_Input 4곳 전부 수정
2. Panel이 구체 탭 헤더를 전부 include → 컴파일 의존성 증가
3. m_pCurTab 같은 단일 포인터로 다룰 수 없음 (타입이 다르므로 한 컨테이너에 못 담음)

Slide 4-1. 왜 `unique_ptr` 인가

```
vector<unique_ptr<IAttributeTab>> m_vecTabs;    // 유일한 소유자
IAttributeTab* m_pCurTab;    // 관찰용, 소유권 없음
```



- `unique_ptr` = "이 객체는 오직 나만 소유한다"를 타입으로 강제
- 벡터가 파괴되면 안의 탭 3개도 자동 해제 (RAII)
- `m_pCurTab` 에 `delete` 하면 이중 해제 → raw pointer로 둔 것이 의도

Slide 4-2. & 와 &&

```
void f(vector<int>& v);    // 좌측값 참조 - 이름 있는 변수를 받음
void f(vector<int>&& v);    // 우측값 참조 - 임시값 / std::move된 것만 받음
```



용어	뜻	예
좌측값(lvalue)	이름이 있어 나중에 또 쓸 수 있는 것	tabs
우측값(rvalue)	곧 사라질 임시값	Create() 의 반환값, std::move(x)

`&&` 는 `&` 두 개가 아니라 별개의 토큰이다. (`*` → `**` 는 진짜 2단 포인터지만, `&` → `&&` 는 그런 관계가 아님)

Slide 4-3. `std::move` 는 이동하지 않는다

```
template<typename T>
T&& move(T& x) { return static_cast<T&&>(x); } // 그냥 캐스팅
```



실제 이동은 그 다음 대입에서 일어난다. `vector` 에는 대입 연산자가 2개 있다.

```
vector& operator=(const vector& other); // ① 복사 대입
vector& operator=(vector&& other); // ② 이동 대입
```



코드	값 범주	선택되는 함수	결과
<code>m_vecTabs = vecTabs;</code>	좌측값	① 복사	<code>unique_ptr</code> 복사 시도 → 컴파일 에러
<code>m_vecTabs = std::move(vecTabs);</code>	우측값	② 이동	포인터만 이전 → 성공

① 복사: 새 배열 할당 + 원소 하나하나 복사 → `unique_ptr` 은 복사 불가 → 에러
② 이동: `m_pData` 포인터만 훔쳐오고 상대를 `nullptr` 로 → 원소를 안 건드림 → OK

Slide 4-4. 왜 함수 안에서 또 `std::move`?

```
void Set_AttributeTabs(vector<unique_ptr<IAttributeTab>>&& vecTabs)   
{  
    m_vecTabs = vecTabs;           // ✕ 에러  
    m_vecTabs = std::move(vecTabs); // ○ OK  
}
```

규칙: 이름이 붙는 순간 좌측값이다. 타입이 `&&` 여도 마찬가지.

위치	<code>std::move</code> 가 하는 일	없으면
호출부	인자가 <code>&&</code> 파라미터에 바인딩되도록	"우측값 참조에 좌측값을 붙일 수 없음" 에러
구현부	이동 대입 연산자가 선택되도록	복사 대입이 선택 → <code>unique_ptr</code> 복사 불가 에러

```
// ① 호출부 - tabs는 이름 있는 변수(좌측값)  
panel->Set_AttributeTabs(std::move(tabs));  
// └─ 이게 없으면 함수 호출 자체가 안 됨
```

```
// ② 구현부 - vecTabs도 이름이 붙어서 좌측값  
void Set_AttributeTabs(vector<unique_ptr<IAttributeTab>>&& vecTabs)  
{  
    m_vecTabs = std::move(vecTabs);  
    // └─ 이게 없으면 복사 대입이 골라져서 에러  
}
```

현재

```
// Select_Cube_1P (1085) - 지역 변수에 담이 재사용
auto* pCubeTab = static_cast<CAttributeTab_Cube*>(
    m_pAttributePanel->Get_Tab(CSelect_AttributePanel::TAB_ORDER::CUBE));
m_tBattle_Level_Desc.iLeftCharCube = pCubeTab->Get_SelectedCubeType();
pCubeTab->Set_CubeSelected();

// Select_Cube_2P (1121-1122) - 같은 캐스팅을 두 번 반복
m_tBattle_Level_Desc.iRightCharCube = static_cast<CAttributeTab_Cube*>(
    m_pAttributePanel->Get_Tab(...TAB_ORDER::CUBE))->Get_SelectedCubeType();
static_cast<CAttributeTab_Cube*>(
    m_pAttributePanel->Get_Tab(...TAB_ORDER::CUBE))->Set_CubeSelected();
```

이때 대상 클래스가 확실해서
dynamic_cast 대신 static_cast 썼다.

If else로 바꾼다면

```
// Level_Select.h
unique_ptr<CAttributeTab_Cube>      m_pCubeTab;
unique_ptr<CAttributeTab_CharColor> m_pCharColorTab;
unique_ptr<CAttributeTab_Helper>    m_pHelperTab;
```

```
void CSelect_AttributePanel::Update(_float fTimeDelta)
{
    if (m_iCurTabIndex == 0) m_pCubeTab->Update(fTimeDelta);
    else if (m_iCurTabIndex == 1) m_pCharColorTab->Update(fTimeDelta);
    else if (m_iCurTabIndex == 2) m_pHelperTab->Update(fTimeDelta);
}
// Late_Update, Render, Handle_Input에도 똑같은 분기 → 총 4곳
```

캐스팅이 사라진당.

```
// Before (현재)
auto* pCubeTab = static_cast<CAttributeTab_Cube*>(Get_Tab(TAB_ORDER::CUBE));
m_tBattle_Level_Desc.iLeftCharCube = pCubeTab->Get_SelectedCubeType();

// After (if/else)
m_tBattle_Level_Desc.iLeftCharCube = m_pCubeTab->Get_SelectedCubeType();
```



<- 분기마다 돌려야 되긴 한다.